DHAR

Bare Metal. Absolute Control.

VOLUME II — ENGINEERING ROADMAP

STAGE-GAMMA

"Optimizing Dhar — earning the C-class performance commitment."

DOCUMENT	Stage-gamma — Optimizing Dhar
VERSION	1.1
DATE	October 26, 2026
STATUS	Ratified
PART OF	The Canon of Dhar, Volume II
DEPENDS ON	Spec0, Spec1, Stage-beta (self-hosting proven), Amendment A-1
PRECEDES	Stage-delta — Gearing Dhar

1. Mandate

Turn Stage-beta's correct-but-naive code generation into an optimizing compiler that aims to stand behind the near-hardware, at-least-C-class performance commitment in Spec0 §2.2. The aim is stated here; the evidence for it is gate A4 (Amendment A-1 §9): the gap to optimised C (`-O3`) on matrix multiplication and on the reference network is measured on the same machine, recorded with its method, and published as it stands, whatever the ratio is. Stage-gamma must not begin until Stage-beta is genuinely self-hosting (Stage-beta §4) — optimizing an unproven compiler produces unproven optimizations.

Dhar is a language for trustworthy computation: it goes first to cybersecurity, then to artificial intelligence, then to quantum computing, with every other domain kept as modules (Spec0 §5.1; Amendment A-1). For Stage-gamma that has one consequence above all: **an optimizer must never make secure code insecure.** Speed that costs the constant-time guarantee is not accepted (§3.1).

2. Scope

2.1 In scope

CATEGORY	REQUIRED PASSES
Representation	SSA construction
Register allocation	Graph-coloring allocation (gate A4 waits for it: Amendment A-1 §7)
Classical optimizations	Constant propagation, dead-code elimination, loop-invariant code motion, GVN/PRE
Interprocedural	Inlining, devirtualization of forge method calls (Spec1 §10.2)
Backend-facing	Vectorization (including the fixed-width vector types of Spec1 §4.7 on the vector unit of each target), jump-table lowering for choose/is (Spec1 §8.3), architecture-specific tuning

Every pass above keeps the invariants of §3.1.

2.2 Out of scope

- New language features or syntax changes — Spec1 is frozen for the duration of this stage.
- New back ends beyond what is needed to demonstrate the optimizations; the back ends of every committed target are Stage-beta's (Stage-beta §4.2), and a target outside Amendment A-2 §3.1 is added by revising that amendment.
- Accelerators (GPUs and similar) and model training, until gate A4 is met (Amendment A-1 §5).

3. Required Architecture

Stage-gamma operates on Stage-beta's Universal IR, not on source text or the AST directly. This preserves the boundary Spec0 §4 establishes between architecture-agnostic and architecture-specific concerns: an optimization pass that needs to inspect source syntax rather than IR structure is almost certainly misplaced.

```
Stage-beta AST
      |
      v
Universal IR ----> [ Stage-gamma optimization passes operate here ]
      |
      v
Backend emission (unchanged interface from Stage-beta)
```

3.1 Invariants every pass keeps

1. **Constant time is preserved** (Amendment A-1, R2 and S2). A value or a memory region the IR marks secret (Spec1 §14.5; Stage-beta §3.1, R2) is never made to decide a branch, an address or an instruction of variable time by any pass. In particular, no pass may turn a select into a branch on a secret, introduce a table lookup indexed by a secret, or delete, shorten or reorder the clearing of a secret buffer. The passes this constrains are constant propagation and folding, strength reduction, if-conversion, loop transforms (including loop-invariant code motion), inlining, GVN/PRE (which may merge a secret and a public value), dead-code elimination (which may not remove a clearing as dead) and vectorization.
2. **Floating point is deterministic by default** (Amendment A-1, A1; Spec1 §4.7). No pass reorders, reassociates or contracts floating-point operations, and none changes the order of a reduction, unless the program asked for it by name.
3. **A target changes the encoding, not the meaning** (Amendment A-2 §8). An optimization produces, on every committed target, a program with the same observable behaviour as the unoptimized one, and the back end of every committed target keeps passing the conformance battery after optimization (Stage-alpha §3.3).
4. **Each invariant has a test per pass.** For invariant 1, the test is a program with a secret, the pass applied alone, and the emitted code inspected for a secret-dependent branch or table lookup and for the clearing, with the timing test of gate S2 (Amendment A-1 §9) run on the result.

4. Acceptance Criteria

1. Correctness is non-negotiable: every test passing at the end of Stage-beta must still pass after every individual optimization pass is enabled, not only after the full pipeline runs together.
2. Each pass must be independently toggleable, so that a regression can be isolated to a single pass rather than the optimizer as a whole.
3. Each pass added must show a measured improvement on the language's own benchmark suite (Spec0 §7) relative to the baseline without it.
4. A numeric performance target relative to C is deliberately not fixed in this document: it should be set once Stage-beta's own unoptimized baseline is measured, and recorded here in a future revision rather than invented now without a baseline to invent it against. Gate A4 requires that the ratio to optimised C on the A4

kernels be measured, recorded with its method and published as it stands.

5. Every pass has the constant-time test of §3.1, invariant 4, and no pass changes the result of a program that the secret-value tests of Spec1 §14.5 accept or reject.
6. The floating-point edge-value tests of gate A1 (Amendment A-1 §9) give bit-identical results with every pass enabled.
7. The conformance battery of every committed target passes with every pass enabled (§3.1, invariant 3).

5. Handoff to Stage-delta

Stage-delta may begin once Stage-gamma has landed SSA construction, register allocation, and dead-code elimination — the minimum viable optimizing pipeline — even if the remaining passes are still in progress. Stage-gamma and Stage-delta are not as strictly sequential as Stage-alpha, Stage-beta, and Stage-gamma are with each other.

Revision log

Stage-gamma — Optimizing Dhar · v1.1 · October 26, 2026 · Ratified.

VERSION	DATE	CHANGE
v1.0	September 6, 2026	First ratified edition; performance target intentionally left open pending a real Stage-beta baseline.
v1.1	October 26, 2026	Amendment A-1: the constant-time-preserving rule as an invariant of every pass, with a test per pass (§3.1); deterministic floating point by default; the performance commitment worded as an aim whose evidence is gate A4 (§1, §4).